SOC Detection Master Pack

Sysmon lab, password reset chain, 35 detections, an attack chain tracker, and gap fixes

Read this first

- None of these queries were run on a live Splunk environment while writing this. Treat every one as a draft. Test it by creating the behaviour in a lab (Atomic Red Team has ready-made tests per technique), and confirm it also stays quiet on a normal day.
- Field names differ between Splunk setups. Windows events can show Account_Name or TargetUserName, Source_Network_Address or IpAddress, and so on. Run B1 and B2 first, then edit the queries to match what you see. Many queries use coalesce() to cope with both.
- Sysmon queries use sourcetype XmlWinEventLog:Microsoft-Windows-Sysmon/Operational. If yours is different, change it. Replace index=* with your real index.
- Several queries need extra logging turned on: Sysmon with EventCode 10 for lsass, 4688 process creation, Directory Service Access for DCSync, Other Object Access for scheduled tasks. A silent query may just mean the logging is off.
- Thresholds (5 services, 10 users, 20 connections) are starting points. Tune them to your environment.
- If you can only turn on a few alerts first, start with: B10, D1, C1, C3, C4, C5, C8, C9, C10, C13, C17, C18.
- Part E closes gaps found while testing: E1 fixes D1 for processes with no parent GUID, E2 covers the runas hop in the password chain, E3 scopes an incident across hosts, E4 catches net user /add, E5 is a one-host storyline.
- D1 only tracks processes linked by ParentProcessGuid. Processes whose parent GUID is all zeros (as happened with net user /add in the lab) cannot be attached to a chain. Sysmon also has no rename event, so renames are found by command line or by matching the SHA256 hash, which needs EventCode 15 or 29 data.
- For a much bigger tested library, use Splunk Security Essentials or the Enterprise Security Content Update app, the Splunk Security Content repository (github.com/splunk/security_content), and the detection pages at research.splunk.com. SigmaHQ rules and Atomic Red Team tests are the other two free sources people use.
- A hit is a lead, not a verdict. Always check the raw event and ask whether the activity has a normal explanation.
- Copy queries from the .md file or from GitHub, not from this PDF. Copying from a PDF can break line endings and quote marks.

Contents

A1 Trojan started cmd.exe (when?)

A2 New user added with net user /add

A3 Script file dropped on disk (.ps1)

A4 Firewall rule added

A5 Process chain (parent and child) by ProcessGuid

A6 Network connections made by one process

B1 Check the data exists

B2 Check the field names

B3 How many users changed their own password (exact name match)

B4 Same, treating firstname_xxx accounts as one person

B5 Discover which account name pairs exist

B6 Entra ID (Azure AD) self-service password reset

B7 Okta password reset or update

B8 Raw timeline for one user on one day (abc*)

B9 Prove the password change came from the jump box (domain controller side)

B10 MAIN DETECTION: RDP logon, then password change/reset, then target account logs in

B11 One line per incident (replace the last two lines of B10)

C1 LSASS memory access (credential dumping)

C2 LSASS dump using debug DLLs

C3 Kerberoasting (RC4 service ticket requests)

C4 DCSync (fake domain controller replication)

C5 Password spraying (one source, many users)

C6 Many failures then a success (brute force worked)

C7 Account lockouts

C8 New user account created

C9 User added to a privileged group

C10 New service installed with a suspicious path

C11 Scheduled task created with a suspicious command

C12 Run key persistence

C13 Encoded PowerShell

C14 PowerShell download or hiding flags

C15 Office app starting a shell or script engine

C16 Living-off-the-land binaries (certutil, bitsadmin, mshta, regsvr32)

C17 Shadow copies or backups deleted (ransomware step)

C18 Event log cleared

C19 Defender or antivirus tampering

C20 RDP logon from a public IP address

C21 One account logging into many hosts quickly

C22 PsExec style remote execution

C23 Burst of recon commands (whoami, tasklist and similar)

C24 Domain and account enumeration commands

C25 Beaconing (regular outbound connections)

C26 Shells and script tools making outside connections

C27 DNS tunnelling or odd DNS volume

C28 Programs running from user-writable folders

C29 Double-extension programs (invoice.pdf.exe)

C30 Programs seen for the first time in the last day

C31 Linux SSH brute force

C32 Linux account created or changed

C33 Entra ID privileged role assignment (keyword hunt)

C34 Entra ID application consent (illicit consent grant)

C35 Web attack patterns (SQL injection)

D1 MAIN: file dropped or downloaded, renamed, executed, and everything it spawned (parent and child IDs)

D2 Same chain, one line per incident (replace the last three lines of D1)

D3 Drill into one chain by its root GUID

D4 SAMPLE RESULT (real lab data, not Splunk output)

E1 Rescue processes with no parent GUID (fixes the gap in D1)

E2 Account hop with runas (completes the B10 password chain)

E3 Scope the incident: who else touched this IP, hash or file name

E4 Account creation or local admin added from the command line

E5 Storyline for one host (what happened, in time order)

Part A - Sysmon investigation (lab file sysmon.json)

A1. Trojan started cmd.exe (when?)

Finds the process-create event (EventID 1) where cmd.exe was started, with its parent. Replace YOUR-HOST with the host name.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=1 "Event.EventData.Image"="*\\cmd.exe"
| sort + _time
| table _time Event.EventData.ParentImage Event.EventData.Image Event.EventData.CommandLine
```

A2. New user added with net user /add

Shows net.exe / net1.exe process creations, including the username and password typed on the command line.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=1
("Event.EventData.Image"="*\\net.exe" OR "Event.EventData.Image"="*\\net1.exe")
| sort + _time
| table _time Event.EventData.Image Event.EventData.CommandLine Event.EventData.ParentImage
```

A3. Script file dropped on disk (.ps1)

EventID 11 is file create. The NOT filter removes the harmless __PSScriptPolicyTest files that PowerShell creates by itself.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=11 "Event.EventData.TargetFilename"="*.ps1"
NOT "Event.EventData.TargetFilename"="*__PSScriptPolicyTest*"
| sort + _time
| table _time Event.EventData.Image Event.EventData.TargetFilename Event.EventData.ProcessGuid
```

A4. Firewall rule added

Sysmon does not log firewall rules as their own event. They show up as registry value sets (EventID 13) under FirewallRules. RA4= marks a rule with a remote IPv4 address.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=13 "Event.EventData.TargetObject"="*FirewallRules*"
"Event.EventData.Details"="*RA4=*"
| sort - _time
| table _time Event.EventData.User Event.EventData.Image Event.EventData.Details Event.EventData.TargetObject
```

A5. Process chain (parent and child) by ProcessGuid

Use ProcessGuid, not the process name, to follow one exact process. Replace YOUR-GUID with the ProcessGuid from the event you are tracing.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
("Event.EventData.ProcessGuid"="YOUR-GUID" OR "Event.EventData.ParentProcessGuid"="YOUR-GUID")
| sort + _time
| table _time Event.System.EventID Event.EventData.ParentImage Event.EventData.Image Event.EventData.CommandLine
      Event.EventData.TargetFilename
```

A6. Network connections made by one process

EventID 3 is a network connection. Shows where a suspicious process connected to.

```
source="sysmon.json" host="YOUR-HOST" sourcetype="_json"
"Event.System.EventID"=3 "Event.EventData.ProcessGuid"="YOUR-GUID"
| table _time Event.EventData.SourceIp Event.EventData.DestinationIp Event.EventData.DestinationPort
```

Part B - Password reset / jump box investigation (Windows Security logs)

B1. Check the data exists

Run this first. It confirms the password and logon events are being ingested, and which index and host they come from.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724 OR EventCode=4624)
| stats count by index host EventCode
```

B2. Check the field names

Run this second. If any column comes back empty, that field has a different name in your logs and the later queries need changing to match.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724 OR EventCode=4624)
| head 20
| table _time host EventCode Account_Name Subject_Account_Name Target_Account_Name Logon_Type
      Source_Network_Address Workstation_Name Caller_Computer_Name
```

B3. How many users changed their own password (exact name match)

4723 = an account changed its own password. 4724 = someone reset it. Self change means the account doing it and the account affected are the same.

```
index=* sourcetype=WinEventLog:Security EventCode=4723
| where lower(Subject_Account_Name)=lower(Target_Account_Name)
| stats dc(Target_Account_Name) as users_who_changed_own_password, count as total_changes
```

B4. Same, treating firstname_xxx accounts as one person

Cuts everything after the first underscore, so manish_backup and manish_svc both become manish. Includes 4724 because changing another account of your own shows up as a reset.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724)
| eval subj=lower(Subject_Account_Name), targ=lower(Target_Account_Name)
| eval subj_base=replace(subj,"_.*$",""), targ_base=replace(targ,"_.*$","")
| where subj_base=targ_base
| stats dc(targ_base) as users_who_reset_own_password, count by EventCode
```

B5. Discover which account name pairs exist

Lists every case where the account doing the change differs from the account affected. Use it to see which suffixes really exist in your environment.

```
index=* sourcetype=WinEventLog:Security (EventCode=4723 OR EventCode=4724)
| eval subj=lower(Subject_Account_Name), targ=lower(Target_Account_Name)
| where subj!=targ
| stats count by subj targ
| sort - count
```

B6. Entra ID (Azure AD) self-service password reset

Use this if resets happen through the Microsoft self-service portal instead of Windows events.

```
index=* sourcetype="azure:aad:audit"
("Self-service password reset flow activity progress" OR "Reset password (self-service)")
| stats dc(targetResources{}.userPrincipalName) as users, count
```

B7. Okta password reset or update

Use this if the company uses Okta.

```
index=* sourcetype="OktaIM2:log"
(eventType="user.account.reset_password" OR eventType="user.account.update_password")
| stats dc(actor.alternateId) as users, count
```

B8. Raw timeline for one user on one day (abc*)

Shows the real events one after another: RDP logon, explorer.exe start, then the password event. No jump box name needed, the host column shows the machine. Dates are month/day/year, so 10/11/2026 is 11 October.

```
index=* sourcetype=WinEventLog:Security earliest="10/11/2026:00:00:00" latest="10/12/2026:00:00:00"
  (
    ((EventCode=4723 OR EventCode=4724) (Subject_Account_Name="abc*" OR Target_Account_Name="abc*"))
    OR (EventCode=4624 Logon_Type=10 Account_Name="abc*")
    OR (EventCode=4688 New_Process_Name="*\\explorer.exe" Account_Name="abc*")
  )
| sort 0 _time
| table _time host EventCode Subject_Account_Name Target_Account_Name Account_Name New_Process_Name Logon_Type
      Source_Network_Address Caller_Computer_Name _raw
```

B9. Prove the password change came from the jump box (domain controller side)

Password events are logged on the domain controller. This matches each password event to the network logon (type 3) that carried it, using the Logon ID on the same host. The Source_Network_Address or Workstation_Name of that logon should be the jump box.

```
index=* sourcetype=WinEventLog:Security earliest=-24h
  ((EventCode=4723 OR EventCode=4724) OR (EventCode=4624 Logon_Type=3))
| eval sess=if(EventCode=4624, mvindex(Logon_ID,-1), mvindex(Logon_ID,0))
| eventstats values(EventCode) as codes by host sess
| where mvfind(codes,"4624")>=0 AND (mvfind(codes,"4723")>=0 OR mvfind(codes,"4724")>=0)
| sort 0 host sess _time
| table _time host EventCode sess Account_Name Source_Network_Address Workstation_Name Logon_Type _raw
```

B10. MAIN DETECTION: RDP logon, then password change/reset, then target account logs in

Finds the full chain for any user, with no account suffix and no jump box name in the search. Step 1 is an RDP logon by an account that never has its password changed. Step 2 is a password change or reset where both accounts share the first name. Step 3 is the changed account logging in afterwards. The host column on step 1 is the machine that was used. Not tested on your Splunk: try it on a known case first.

```
index=* sourcetype=WinEventLog:Security earliest=-24h
  ((EventCode=4723 OR EventCode=4724) OR (EventCode=4624 (Logon_Type=2 OR Logon_Type=3 OR Logon_Type=10 OR
       Logon_Type=11)))
| eval acct=lower(if(EventCode=4624, mvindex(Account_Name,-1), Target_Account_Name))
| eval subj=lower(Subject_Account_Name)
| where match(acct,"_")
| eval user_base=replace(acct,"_.*$","")
| eval is_pw=if((EventCode=4723 OR EventCode=4724) AND subj!=acct AND replace(subj,"_.*$","")=user_base,1,0)
| eval is_logon=if(EventCode=4624,1,0)
| where is_pw=1 OR is_logon=1
| eval t_pw=if(is_pw=1,_time,null())
| eventstats min(t_pw) as pw_on_acct by acct
| eval is_rdp=if(is_logon=1 AND Logon_Type=10 AND isnull(pw_on_acct),1,0)
| eval stage=case(is_rdp=1,1, is_pw=1,2, is_logon=1 AND isnotnull(pw_on_acct) AND _time>pw_on_acct,3)
| where isnotnull(stage)
| eval t_rdp=if(stage=1,_time,null()), t3=if(stage=3,_time,null())
| eventstats min(t_rdp) as first_rdp min(t_pw) as first_pw max(t3) as last_login by user_base
| where isnotnull(first_rdp) AND isnotnull(first_pw) AND isnotnull(last_login) AND first_rdp<first_pw AND
      last_login>first_pw
| eval step=case(stage=1,"1_RDP_logon", stage=2 AND EventCode=4723,"2_password_CHANGED", stage=2 AND
      EventCode=4724,"2_password_RESET", stage=3,"3_target_account_login")
| sort 0 user_base _time
| table _time host user_base step EventCode acct Subject_Account_Name Target_Account_Name Logon_Type
      Source_Network_Address Workstation_Name Caller_Computer_Name
```

B11. One line per incident (replace the last two lines of B10)

Delete the final sort and table lines of B10 and put these on the end instead. Better for a saved alert, since each incident is one row.

```
| stats earliest(_time) as start, latest(_time) as end, values(acct) as accounts_involved,
      values(Subject_Account_Name) as changed_by, values(host) as hosts, values(Source_Network_Address) as
      source_ips by user_base
| convert ctime(start) ctime(end)
```

Part C1 - Credential access (stealing or guessing passwords)

C1. LSASS memory access (credential dumping)

MITRE T1003.001. Sysmon EventCode 10 where a process opens lsass.exe with the access rights Mimikatz-style tools use (0x1010, 0x1410). Based on a published Splunk Security Content detection. Needs Sysmon logging EventCode 10 for lsass.exe. Tune: Splunk warns about false positives, so exclude known AV/EDR SourceImage values.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=10 TargetImage="*\\lsass.exe"
(GrantedAccess=0x1010 OR GrantedAccess=0x1410)
| stats count min(_time) as firstTime max(_time) as lastTime by Computer, SourceImage, SourceProcessId,
      TargetImage, GrantedAccess
| convert ctime(firstTime) ctime(lastTime)
```

C2. LSASS dump using debug DLLs

MITRE T1003.001. Catches tools that dump LSASS memory by calling dbgcore.dll or dbghelp.dll (the call that writes a minidump). Pairs well with C1.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=10 TargetImage="*\\lsass.exe"
(CallTrace="*dbgcore.dll*" OR CallTrace="*dbghelp.dll*")
| stats count by Computer, SourceImage, GrantedAccess, CallTrace
| sort - count
```

C3. Kerberoasting (RC4 service ticket requests)

MITRE T1558.003. Event 4769 with RC4 encryption (0x17) for a normal user service, then a burst of different services from one requester. Based on a published Splunk detection, plus a burst count. Run on domain controller logs. Field names: Splunk's content uses ServiceName and TicketEncryptionType; classic format may show Service_Name and Ticket_Encryption_Type, so run B2 first. Tune: raise or lower the 5. Modern attackers may use AES, so this misses those.

```
index=* sourcetype=*WinEventLog:Security EventCode=4769 TicketEncryptionType=0x17 ServiceName!="*$"
      ServiceName!="krbtgt"
| bin _time span=10m
| stats dc(ServiceName) as unique_services values(ServiceName) as services count by _time, IpAddress, TargetUserName
| where unique_services>=5
```

C4. DCSync (fake domain controller replication)

MITRE T1003.006. Event 4662 where a non-computer account uses the directory replication rights (the GUIDs are DS-Replication-Get-Changes and Get-Changes-All). Needs auditing of Directory Service Access on domain controllers. Tune: build a list of legitimate replication accounts such as Azure AD Connect and exclude them.

```
index=* sourcetype=*WinEventLog:Security EventCode=4662
(Properties="*1131f6aa-9c07-11d1-f79f-00c04fc2dcd2*" OR Properties="*1131f6ad-9c07-11d1-f79f-00c04fc2dcd2*" OR
      Properties="*89e95b76-444d-4c62-991a-0facbeda640c*")
| eval subject=coalesce(SubjectUserName, Subject_Account_Name, Account_Name)
| where NOT like(subject,"%$")
| stats count min(_time) as firstTime max(_time) as lastTime by host, subject
| convert ctime(firstTime) ctime(lastTime)
```

C5. Password spraying (one source, many users)

MITRE T1110.003. Many different usernames failing from one source in 5 minutes. Based on Splunk's published idea (failed network logons, Event 4625 type 3). Splunk's own version uses 30 users, this uses 10 so test and tune the number. Needs Audit Logon enabled.

```
index=* sourcetype=*WinEventLog:Security EventCode=4625 Logon_Type=3
| eval src=coalesce(IpAddress, Source_Network_Address), tuser=lower(coalesce(TargetUserName,
      mvindex(Account_Name,-1)))
| where isnotnull(src) AND src!="-"
| bin _time span=5m
| stats dc(tuser) as unique_users values(tuser) as users count by _time, src
| where unique_users>=10
```

C6. Many failures then a success (brute force worked)

MITRE T1110. The same source and user with 10 or more failed logons and at least one success in the same 15 minute window. A success after failures is the one to look at first.

```
index=* sourcetype=*WinEventLog:Security (EventCode=4625 OR EventCode=4624)
| eval src=coalesce(IpAddress, Source_Network_Address), tuser=lower(coalesce(TargetUserName,
      mvindex(Account_Name,-1)))
| where isnotnull(src) AND src!="-" AND src!="::1" AND src!="127.0.0.1"
| bin _time span=15m
| stats count(eval(EventCode=4625)) as fails, count(eval(EventCode=4624)) as successes by _time, src, tuser
| where fails>=10 AND successes>=1
```

C7. Account lockouts

MITRE T1110. Event 4740. Shows which accounts are being locked out and which computer the bad attempts came from.

```
index=* sourcetype=*WinEventLog:Security EventCode=4740
| eval locked=coalesce(TargetUserName, Account_Name), from_host=coalesce(CallerComputerName, Caller_Computer_Name)
| stats count values(from_host) as lockout_source by locked
| sort - count
```

Part C2 - Persistence and privilege escalation (staying in, getting more power)

C8. New user account created

MITRE T1136. Event 4720. Review every hit: who created it, on which host, and does it match a real request.

```
index=* sourcetype=*WinEventLog:Security EventCode=4720
| eval creator=coalesce(SubjectUserName, Subject_Account_Name), new_account=coalesce(TargetUserName,
      Target_Account_Name)
| table _time host creator new_account _raw
```

C9. User added to a privileged group

MITRE T1098. Events 4728, 4732, 4756 (member added to a global, local or universal group) that mention a powerful group. Edit the group list for your environment.

```
index=* sourcetype=*WinEventLog:Security (EventCode=4728 OR EventCode=4732 OR EventCode=4756)
("Administrators" OR "Domain Admins" OR "Enterprise Admins" OR "Schema Admins" OR "Backup Operators" OR "Account
      Operators")
| table _time host EventCode Subject_Account_Name Group_Name Member_Name _raw
```

C10. New service installed with a suspicious path

MITRE T1543.003. System event 7045 or Security event 4697, kept only if the service points at Temp, ProgramData, Public, or a shell. PsExec-style tools and malware both do this.

```
index=* ((sourcetype=WinEventLog:System EventCode=7045) OR (sourcetype=*WinEventLog:Security EventCode=4697))
| regex _raw="(?i)(\\\\Temp\\\\|\\\\ProgramData\\\\|\\\\Users\\\\Public\\\\|cmd\.exe|powershell|rundll32|mshta)"
| table _time host EventCode _raw
```

C11. Scheduled task created with a suspicious command

MITRE T1053.005. Event 4698, kept only if the task runs a shell, script engine or a file in a temp style folder. Needs Audit Other Object Access Events enabled.

```
index=* sourcetype=*WinEventLog:Security EventCode=4698
| regex
      _raw="(?i)(powershell|cmd\.exe|wscript|cscript|mshta|rundll32|regsvr32|\\\\Temp\\\\|\\\\ProgramData\\\\|\\\\Us
      ers\\\\Public\\\\)"
| table _time host Subject_Account_Name Task_Name _raw
```

C12. Run key persistence

MITRE T1547.001. Sysmon EventCode 13 writing to the Run or RunOnce registry keys. Expect a lot of normal software at first, so baseline for a week and then alert on new entries.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=13
(TargetObject="*\\CurrentVersion\\Run\\*" OR TargetObject="*\\CurrentVersion\\RunOnce\\*")
| stats count min(_time) as first max(_time) as last by Computer, Image, TargetObject, Details, User
| convert ctime(first) ctime(last)
```

Part C3 - Execution and defence evasion (running code, hiding, breaking defences)

C13. Encoded PowerShell

MITRE T1059.001. PowerShell run with an encoded command (the flag can be abbreviated, so this catches -e, -enc, -EncodedCommand and the rest).

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
(Image="*\\powershell.exe" OR Image="*\\pwsh.exe")
| regex CommandLine="(?i)\s-(e|ec|en|enc|enco|encod|encode|encoded|encodedcommand)\s+[A-Za-z0-9+/=]{20,}"
| table _time Computer User ParentImage Image CommandLine
```

C14. PowerShell download or hiding flags

MITRE T1059.001, T1105. Download cradles, hidden windows and execution policy bypass. In your lab, the attacker used -ep bypass.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
(Image="*\\powershell.exe" OR Image="*\\pwsh.exe")
| regex
      CommandLine="(?i)(downloadstring|downloadfile|invoke-webrequest|\biwr\b|invoke-expression|\biex\b|net\.webclie
      nt|-nop|-noprofile|-w\s+hidden|-windowstyle\s+hidden|-ep\s+bypass|-executionpolicy\s+bypass)"
| table _time Computer User ParentImage CommandLine
```

C15. Office app starting a shell or script engine

MITRE T1204.002, T1566. Word, Excel, PowerPoint, Outlook or Access launching cmd, PowerShell, wscript, mshta and similar. Very low false positives, so a strong alert.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
ParentImage IN ("*\\winword.exe","*\\excel.exe","*\\powerpnt.exe","*\\outlook.exe","*\\msaccess.exe")
Image IN
      ("*\\cmd.exe","*\\powershell.exe","*\\pwsh.exe","*\\wscript.exe","*\\cscript.exe","*\\mshta.exe","*\\rundll32.
      exe","*\\regsvr32.exe")
| table _time Computer User ParentImage Image CommandLine
```

C16. Living-off-the-land binaries (certutil, bitsadmin, mshta, regsvr32)

MITRE T1218, T1105. Built-in Windows tools being used to download or run code.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| regex
      CommandLine="(?i)(certutil.*(-urlcache|-decode|-encode)|bitsadmin.*/transfer|mshta.*(http|vbscript|javascript)
      |regsvr32.*(/i:http|scrobj)|rundll32.*javascript)"
| table _time Computer User ParentImage Image CommandLine
```

C17. Shadow copies or backups deleted (ransomware step)

MITRE T1490. Commands that wipe Windows backups. Treat any hit as urgent.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| regex
      CommandLine="(?i)(vssadmin.*delete\s+shadows|wmic.*shadowcopy.*delete|bcdedit.*(recoveryenabled\s+no|bootstatu
      spolicy\s+ignoreallfailures)|wbadmin.*delete\s+(catalog|backup))"
| table _time Computer User ParentImage Image CommandLine
```

C18. Event log cleared

MITRE T1070.001. Security log cleared (1102) or System log cleared (104). Almost never legitimate on a server.

```
index=* ((sourcetype=*WinEventLog:Security EventCode=1102) OR (sourcetype=*WinEventLog:System EventCode=104))
| eval who=coalesce(SubjectUserName, Account_Name, user)
| table _time host EventCode who _raw
```

C19. Defender or antivirus tampering

MITRE T1562.001. Command lines that disable Defender or add exclusions, plus Defender event 5001 (real-time protection disabled). Check the event ID in Microsoft's documentation for your Windows version.

```
index=* ((sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1) OR
      (sourcetype="WinEventLog:Microsoft-Windows-Windows Defender/Operational" EventCode=5001))
| regex
      _raw="(?i)(set-mppreference.*(disablerealtimemonitoring|disableioavprotection|exclusionpath|exclusionprocess)|
      sc\s+(stop|config)\s+windefend|net\s+stop\s+windefend|Real-Time Protection .* disabled)"
| table _time host EventCode _raw
```

Part C4 - Lateral movement and discovery (spreading, exploring)

C20. RDP logon from a public IP address

MITRE T1021.001. Successful RDP logons (type 10) where the source is not a private address. Should be rare or never.

```
index=* sourcetype=*WinEventLog:Security EventCode=4624 Logon_Type=10
| eval src=coalesce(IpAddress, Source_Network_Address), tuser=coalesce(TargetUserName, mvindex(Account_Name,-1))
| where isnotnull(src) AND src!="-" AND src!="::1" AND src!="127.0.0.1" AND NOT cidrmatch("10.0.0.0/8",src) AND NOT
      cidrmatch("172.16.0.0/12",src) AND NOT cidrmatch("192.168.0.0/16",src)
| table _time host src tuser
```

C21. One account logging into many hosts quickly

MITRE T1021. A user account touching 5 or more different machines in 30 minutes, which is a typical spread pattern. Tune the 5, and exclude scanners and admin tooling.

```
index=* sourcetype=*WinEventLog:Security EventCode=4624 (Logon_Type=3 OR Logon_Type=10)
| eval tuser=lower(coalesce(TargetUserName, mvindex(Account_Name,-1)))
| where NOT like(tuser,"%$") AND tuser!="anonymous logon"
| bin _time span=30m
| stats dc(host) as hosts_touched values(host) as hosts by _time, tuser
| where hosts_touched>=5
```

C22. PsExec style remote execution

MITRE T1569.002, T1021.002. The PSEXESVC service binary being created or run.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
(Image="*\\PSEXESVC.exe" OR ParentImage="*\\PSEXESVC.exe" OR TargetFilename="*\\PSEXESVC.exe")
| table _time Computer EventCode User Image ParentImage CommandLine TargetFilename
```

C23. Burst of recon commands (whoami, tasklist and similar)

MITRE T1033, T1057, T1082. Four or more different discovery tools run by the same user on the same host within 5 minutes. This matches your lab, where the attacker ran whoami then tasklist.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| regex
      Image="(?i)\\\\(whoami|net1?|nltest|ipconfig|tasklist|systeminfo|quser|arp|route|netstat|nslookup|hostname)\.e
      xe$"
| bin _time span=5m
| stats dc(Image) as distinct_tools values(CommandLine) as cmds by _time, Computer, User
| where distinct_tools>=4
```

C24. Domain and account enumeration commands

MITRE T1087, T1069, T1482. Commands that list domain users, groups and controllers.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| regex CommandLine="(?i)(net1?\s+(user|group|localgroup|accounts)\s.*(/domain|domain
      admins|administrators)|nltest\s+/(dclist|domain_trusts)|dsquery|adfind|Get-ADUser|Get-ADGroup)"
| table _time Computer User ParentImage CommandLine
```

Part C5 - Command and control, suspicious files (calling home)

C25. Beaconing (regular outbound connections)

MITRE T1071. Finds a process connecting to the same outside address on a very regular timer, which is how a lot of malware calls home. A small spread of gaps compared with the average gap is the signal. Tune the 20 and the 0.1.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 Initiated=true
| where NOT cidrmatch("10.0.0.0/8",DestinationIp) AND NOT cidrmatch("172.16.0.0/12",DestinationIp) AND NOT
      cidrmatch("192.168.0.0/16",DestinationIp)
| sort 0 Computer Image DestinationIp _time
| streamstats current=f last(_time) as prev_time by Computer Image DestinationIp
| eval delta=_time-prev_time
| stats count avg(delta) as avg_s stdev(delta) as stdev_s by Computer, Image, DestinationIp, DestinationPort
| where count>=20 AND stdev_s<(avg_s*0.1)
```

C26. Shells and script tools making outside connections

MITRE T1071, T1105. PowerShell, mshta, wscript, regsvr32, rundll32 or certutil connecting to a public address.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3
Image IN
      ("*\\powershell.exe","*\\pwsh.exe","*\\mshta.exe","*\\wscript.exe","*\\cscript.exe","*\\regsvr32.exe","*\\rund
      ll32.exe","*\\certutil.exe")
| where NOT cidrmatch("10.0.0.0/8",DestinationIp) AND NOT cidrmatch("172.16.0.0/12",DestinationIp) AND NOT
      cidrmatch("192.168.0.0/16",DestinationIp)
| stats count values(DestinationPort) as ports by Computer, User, Image, DestinationIp
```

C27. DNS tunnelling or odd DNS volume

MITRE T1071.004. Sysmon EventCode 22. A single parent domain with a huge number of unique, long names. Tune the numbers.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=22
| eval qlen=len(QueryName), domain=mvindex(split(QueryName,"."),-2)."."mvindex(split(QueryName,"."),-1)
| stats count dc(QueryName) as unique_names avg(qlen) as avg_len by Computer, domain
| where unique_names>200 AND avg_len>30
```

C28. Programs running from user-writable folders

MITRE T1204, T1036. Processes started from Temp, Public, Downloads or ProgramData. In your lab, application_form.pdf.exe ran from the Downloads folder.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
(Image="*\\AppData\\Local\\Temp\\*" OR Image="*\\Users\\Public\\*" OR Image="*\\Downloads\\*" OR
      Image="*\\ProgramData\\*")
| stats count values(CommandLine) as cmds by Computer, User, Image, ParentImage
| sort - count
```

C29. Double-extension programs (invoice.pdf.exe)

MITRE T1036.007. An executable disguised as a document. Matches the trojan in your lab, application_form.pdf.exe.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| regex Image="(?i)\.(pdf|docx?|xlsx?|pptx?|jpe?g|png|txt)\.(exe|scr|com|bat|js|vbs)$"
| table _time Computer User ParentImage Image CommandLine
```

C30. Programs seen for the first time in the last day

A hunting query rather than an alert. Lists executables that had never run on that host in the previous 30 days. Good for finding a new tool dropped by an attacker.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 earliest=-30d
| stats min(_time) as first_seen count by Computer, Image
| where first_seen>relative_time(now(),"-1d")
| convert ctime(first_seen)
| sort - first_seen
```

Part C6 - Linux, cloud and web

C31. Linux SSH brute force

MITRE T1110. Ten or more failed SSH passwords from one address in 10 minutes. Needs linux_secure (auth.log) data.

```
index=* sourcetype=linux_secure "Failed password"
| rex "Failed password for (invalid user )?(?<user>\S+) from (?<src>\d+\.\d+\.\d+\.\d+)"
| bin _time span=10m
| stats count dc(user) as users values(user) as tried_users by _time, src, host
| where count>=10
```

C32. Linux account created or changed

MITRE T1136.001. useradd, usermod and groupadd messages from the auth log.

```
index=* sourcetype=linux_secure ("useradd" OR "new user" OR "usermod" OR "groupadd")
| table _time host _raw
```

C33. Entra ID privileged role assignment (keyword hunt)

MITRE T1098.003. Someone being given an admin role. Because the field paths vary, this prints the raw event, so expand one to learn your field names (the same method as B2).

```
index=* sourcetype="azure:aad:audit" ("Add member to role" OR "Add eligible member to role" OR "Add app role
      assignment")
| table _time _raw
```

C34. Entra ID application consent (illicit consent grant)

MITRE T1528. A user granting an app access to their data, which phishing campaigns abuse. Keyword hunt, as above.

```
index=* sourcetype="azure:aad:audit" "Consent to application"
| table _time _raw
```

C35. Web attack patterns (SQL injection)

MITRE T1190. SQL injection strings in web access logs. Needs access_combined data, so it will not work on other web log formats without changing the field names.

```
index=* sourcetype=access_combined
| regex
      uri_query="(?i)(union(\s|\+|%20)+select|or(\s|\+|%20)+1=1|sleep\(|benchmark\(|information_schema|xp_cmdshell)"
| stats count values(uri_path) as paths by clientip, status
| sort - count
```

Part D - Attack chain tracker (file dropped, renamed, run, then what it did)

D1. MAIN: file dropped or downloaded, renamed, executed, and everything it spawned (parent and child IDs)

Follows the chain: download or file creation in a user folder, then a rename (caught by command or by matching SHA256 hash), then a process starts from that file, then every child process, outside connection, new file and deletion it causes, linked by ProcessGuid and ParentProcessGuid. Only chains that do something suspicious are shown. Needs Sysmon EventCodes 1, 3, 10, 11, 15, 23 and 29. The logic was re-created in Python and run on the lab sysmon.json (see D4), but it has not been run in Splunk itself, so test it first. For the lab JSON file, add these two lines after the first line: | rename "Event.System.EventID" as EventCode and | rename "Event.EventData.*" as *

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" earliest=-24h
  (EventCode=1 OR EventCode=3 OR EventCode=10 OR EventCode=11 OR EventCode=15 OR EventCode=23 OR EventCode=29)
| eval pguid=lower(replace(ProcessGuid,"[{}]","")), ppguid=lower(replace(ParentProcessGuid,"[{}]",""))
| eval tfile=replace(TargetFilename,":[^\\\\]+$",""), is_stream=if(tfile!=TargetFilename,1,0)
| eval hashfield=coalesce(Hashes,Hash)
| rex field=hashfield "SHA256=(?<sha256>[A-Fa-f0-9]{64})"
| eval sha256=lower(sha256)
| eval
      userpath=if(match(tfile,"(?i)\\\\(Users\\\\[^\\\\]+\\\\(Downloads|Desktop|Documents|AppData)|Users\\\\Public|P
      rogramData|Windows\\\\Temp)\\\\"),1,0)
| rex field=CommandLine
      "(?i)\b(?:ren|rename|move)\s+(?:/y\s+)?\"?(?<old_name>[^\"\s]+)\"?\s+\"?(?<new_name>[^\"\s]+)\"?"
| eval base=if(match(old_name,"[\\\\:]"), replace(old_name,"[^\\\\]+$",""), CurrentDirectory)
| eval new_full=if(EventCode=1 AND isnotnull(new_name), if(match(new_name,"[\\\\:]"), new_name, base.new_name),
      null())
| eval file_event=case(
    EventCode=15 AND is_stream=1 AND match(Contents,"ZoneId=3"), "downloaded_MOTW",
    EventCode=15 AND is_stream=0 AND userpath=1, "file_written",
    EventCode=11 AND userpath=1, "file_created",
    EventCode=29 AND userpath=1, "exe_detected_at_creation",
    EventCode=1 AND isnotnull(new_full), "renamed_by_cmd")
| eval fpath=case(file_event="renamed_by_cmd", lower(new_full), isnotnull(file_event), lower(tfile))
| eval fkey=case(isnotnull(fpath), md5(fpath), EventCode=1, md5(lower(Image)))
| eval hfile=if(userpath=1 AND (EventCode=29 OR (EventCode=15 AND is_stream=0)),1,0)
| eval hkey=if((EventCode=1 OR hfile=1) AND isnotnull(sha256), sha256, null())
| eventstats min(eval(if(isnotnull(file_event),_time,null()))) as f_first, values(file_event) as file_story,
      values(eval(if(file_event="file_created" OR file_event="file_written",Image,null()))) as created_by by fkey
| eventstats min(eval(if(hfile=1,_time,null()))) as h_first, values(eval(if(hfile=1,tfile,null()))) as seen_as by
      hkey
| eval by_name=if(EventCode=1 AND isnotnull(f_first) AND f_first<=_time AND _time-f_first<=3600,1,0)
| eval by_hash=if(EventCode=1 AND isnotnull(h_first) AND h_first<=_time AND _time-h_first<=3600,1,0)
| eval is_dropper=if(by_name=1 OR by_hash=1,1,0)
| eval root=if(is_dropper=1,pguid,null()), depth=if(is_dropper=1,0,null())
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".ppguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2))+1, depth)
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".ppguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
```

```
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2))+1, depth)
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".ppguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2))+1, depth)
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".ppguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2))+1, depth)
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".pguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2)), depth)
| where isnotnull(root)
| eval ext_conn=if(EventCode=3 AND NOT cidrmatch("10.0.0.0/8",DestinationIp) AND NOT
      cidrmatch("172.16.0.0/12",DestinationIp) AND NOT cidrmatch("192.168.0.0/16",DestinationIp) AND NOT
      cidrmatch("127.0.0.0/8",DestinationIp) AND NOT match(DestinationIp,"^(::1|fe80|fc|fd)"),1,0)
| eval new_file=if(EventCode=11,1,0), del_file=if(EventCode=23,1,0)
| eval lsass=if(EventCode=10 AND match(TargetImage,"(?i)\\\\lsass\.exe$"),1,0)
| eval ransom_cmd=if(EventCode=1 AND
      match(CommandLine,"(?i)(vssadmin.*delete\s+shadows|wbadmin.*delete|bcdedit.*recoveryenabled\s+no|cipher\s+/w|w
      evtutil\s+cl)"),1,0)
| eval steal_cmd=if(EventCode=1 AND match(CommandLine,"(?i)(login
      data|logins\.json|key4\.db|\\\\Microsoft\\\\Credentials|ntds\.dit|reg(\.exe)?\s+save\s+hklm\\\\(sam|security|s
      ystem)|Compress-Archive|rclone|curl.*(-T|--upload-file)|Invoke-WebRequest.*-Method\s+Post)"),1,0)
| eval ext=if(EventCode=11, lower(replace(TargetFilename,"^.*\.","")), null())
| eval ext=if(match(ext,"^(tmp|log|dll|dat|etl|pf)$"), null(), ext)
| eventstats dc(TargetFilename) as ext_files by root ext
| eventstats max(ext_files) as r_same_ext_files, sum(new_file) as r_new_files, sum(del_file) as r_deleted_files,
      max(ext_conn) as r_net, max(lsass) as r_lsass, max(ransom_cmd) as r_ransomcmd, max(steal_cmd) as r_steal,
      dc(pguid) as r_procs by root
| fillnull value=0 r_same_ext_files r_new_files r_deleted_files
| eval encrypting=if(r_same_ext_files>=50 OR (r_new_files>=100 AND r_deleted_files>=50),1,0)
| eval score=(r_net*2)+(r_lsass*3)+(r_ransomcmd*3)+(r_steal*2)+(encrypting*3)+if(r_procs>=3,1,0)
| where score>=3
| eval exec_mode=if(depth=0 AND EventCode=1, case(match(ParentImage,"(?i)\\\\explorer\.exe$"),"user_double_click",
      match(ParentImage,"(?i)\\\\(services|taskeng|svchost|wmiprvse)\.exe$"),"service_task_or_wmi",
      true(),"other_process"), null())
| eval how_found=case(by_name=1 AND by_hash=1,"name+hash", by_name=1,"name", by_hash=1,"hash only (renamed or
      copied)")
| eval indent=if(depth>0, substr("................",1,depth*2), ""), tree=indent.mvindex(split(Image,"\\"),-1)
| sort 0 root _time
| table _time host root depth tree EventCode User ppguid pguid ParentProcessId ProcessId ParentImage Image
      CommandLine DestinationIp DestinationPort TargetFilename file_story created_by seen_as exec_mode how_found
      score
```

D2. Same chain, one line per incident (replace the last three lines of D1)

Delete the final eval indent line, sort and table lines from D1 and use these instead. Better for a saved alert.

```
| stats min(_time) as start, max(_time) as end, values(host) as host, dc(pguid) as processes, max(depth) as
      depth_reached, values(eval(if(EventCode=3,DestinationIp.":".DestinationPort,null()))) as outside_connections,
      sum(new_file) as files_created, values(file_story) as file_story, values(seen_as) as seen_as,
      values(exec_mode) as exec_mode, max(score) as score by root
| convert ctime(start) ctime(end)
```

D3. Drill into one chain by its root GUID

Take the root value from D1 and see every event for that process and its direct children. Put the GUID in exactly as it appears in your logs (with braces if your logs have them).

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
("YOUR-ROOT-GUID")
| sort 0 _time
| table _time EventCode ParentProcessGuid ProcessGuid ParentImage Image CommandLine DestinationIp DestinationPort
      TargetFilename
```

D4. SAMPLE RESULT (real lab data, not Splunk output)

This is what D1 finds in the lab sysmon.json. It was produced by re-creating the D1 logic in Python and running it on the lab events, so the columns are shortened and Splunk's own layout will differ. Short IDs are the 4 characters before the trailing zeros of each ProcessGuid.

```
time         d tree                          EC parent>child ppid>pid    what happened
-------------------------------------------------------------------------------------------------------------------
09:28:07.170 0 application_form.pdf.exe       1  6200>D402   4616>1464   started by explorer.exe
      (user_double_click), found by name+hash
09:28:07.299 0 application_form.pdf.exe       3  ->D402      ->1464      OUTBOUND -> 13.232.55.12:30
09:28:55.865 1   cmd.exe                      1  D402>DB02   1464>3520   cmd.exe
09:28:59.037 2     whoami.exe                 1  DB02>DD02   3520>5380   whoami
09:29:22.083 2     tasklist.exe               1  DB02>DF02   3520>688    tasklist
09:40:46.640 2     powershell.exe             1  DB02>C505   3520>8208   powershell.exe
09:40:47.078 2     powershell.exe             11 ->C505      ->8208      FILE CREATED
      ...\Temp\__PSScriptPolicyTest_*.ps1
09:41:43.284 2     powershell.exe             3  ->C505      ->8208      OUTBOUND -> 185.199.108.133:443
09:41:44.015 2     powershell.exe             11 ->C505      ->8208      FILE CREATED C:\Windows\Temp\tmp.ps1
09:42:08.639 3       powershell.exe           1  C505>CC05   8208>6244   powershell.exe -ep bypass
09:42:08.687 3       powershell.exe           11 ->CC05      ->6244      FILE CREATED
      ...\Temp\__PSScriptPolicyTest_*.ps1

Root file       : C:\Users\LetsDefend\Downloads\application_form.pdf.exe
file_story      : downloaded_MOTW, file_created, file_written     created_by : msedge.exe
seen_as (same SHA256 B5BA9F64...): Unconfirmed 994107.crdownload | application_form.pdf.exe | e30f7f00-....tmp
exec_mode       : user_double_click      how_found : name+hash      score : 3 (outside connection 2 + three or more
      processes 1)

Not attached (parent GUID is all zeros, so no GUID link): net.exe 'net user jumpadmin ... /add' at 09:31:57,
      whoami.exe at 09:46:15.
```

Part E - Gaps closed (added after testing the earlier parts)

E1. Rescue processes with no parent GUID (fixes the gap in D1)

D1 cannot attach a process whose ParentProcessGuid is all zeros. In your lab, that left out 'net user jumpadmin ... /add' (09:31:57), net.exe at 09:32:07 and whoami.exe at 09:46:15. I checked the lab data: all three share the trojan's logon session (LogonId 0x1855c) and the same working folder as a process already in the chain. This block attaches such orphans to the chain they match, and then re-runs two child passes so their net1.exe children follow. Insert it in D1 immediately BEFORE the last 'eventstats values(...) as tm' group (the one whose next line uses ppguid replaced by pguid), and add link_note to the final table line. The match is a guess, so link_note says so. Time window is 2 hours. The data conditions were checked on the lab file, but this Splunk code has not been run.

```
| eval is_orphan=if(EventCode=1 AND isnull(root) AND match(ParentProcessGuid,"^\{?0{8}-"),1,0)
| eval lk=if(EventCode=1 AND (isnotnull(root) OR is_orphan=1), md5(LogonId."|".lower(CurrentDirectory)), null())
| eventstats min(eval(if(isnotnull(root),_time,null()))) as lk_first, values(eval(if(isnotnull(root),root,null())))
      as lk_root, min(eval(if(isnotnull(root),depth,null()))) as lk_depth by lk
| eval attach=if(is_orphan=1 AND isnotnull(lk_root) AND _time>=lk_first AND _time-lk_first<=7200,1,0)
| eval root=if(attach=1, mvindex(lk_root,0), root), depth=if(attach=1, lk_depth+1, depth)
| eval link_note=if(attach=1,"GUESSED: no parent GUID, matched by logon session and folder",null())
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".ppguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2))+1, depth)
| eventstats values(eval(if(isnotnull(root),pguid."|".root."|".depth,null()))) as tm
| eval k=mvfind(tm,"^".ppguid."\\|")
| eval root=if(isnull(root) AND isnotnull(k), mvindex(split(mvindex(tm,k),"|"),1), root)
| eval depth=if(isnull(depth) AND isnotnull(k), tonumber(mvindex(split(mvindex(tm,k),"|"),2))+1, depth)
```

E2. Account hop with runas (completes the B10 password chain)

MITRE T1078, T1134. Event 4648 means someone used another account's credentials explicitly, for example runas from manish_admin to manish_backup. This is the missing link in the chain you described: the RDP account becoming the account that changed the password. It shows only hops between accounts that share the first name. Field names vary, so run B2 and adjust.

```
index=* sourcetype=*WinEventLog:Security EventCode=4648
| eval from_acct=lower(coalesce(SubjectUserName, mvindex(Account_Name,0))),
      used_acct=lower(coalesce(TargetUserName, mvindex(Account_Name,1)))
| where from_acct!=used_acct AND replace(from_acct,"_.*$","")=replace(used_acct,"_.*$","") AND NOT
      like(from_acct,"%$")
| table _time host from_acct used_acct ProcessName TargetServerName IpAddress _raw
```

E3. Scope the incident: who else touched this IP, hash or file name

After you find one bad indicator, this shows every host that has it, which one saw it first (likely patient zero), and the last time. Put your indicators in the brackets. Sysmon hash fields contain the SHA256 in capitals, and keyword search is not case sensitive.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" ("13.232.55.12" OR
      "B5BA9F646A7A6A3AECB51B38891B16D1E8B05F063D7B2359C33CF80E36342D18" OR "application_form.pdf.exe")
| stats earliest(_time) as first_seen, latest(_time) as last_seen, count, values(EventCode) as event_types,
      values(Image) as images, values(User) as users by host
| convert ctime(first_seen) ctime(last_seen)
| sort first_seen
```

E4. Account creation or local admin added from the command line

MITRE T1136.001, T1098. Catches net user ... /add and net localgroup administrators ... /add. This would have flagged jumpadmin in your lab: the regex was tested against the lab data and hit both the net.exe and net1.exe lines. The password is hidden in the output column, since the command line contains it in clear text.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| regex
      CommandLine="(?i)\bnet1?(\.exe)?\s+user\s+.*/add|\bnet1?(\.exe)?\s+localgroup\s+(\"?administrators\"?|\"?remot
      e desktop users\"?)\s+.*/add"
| eval cmd_masked=replace(CommandLine,"(?i)(user\s+\S+\s+)\S+(\s+/add)","\1********\2")
| table _time Computer User ParentImage Image cmd_masked
```

E5. Storyline for one host (what happened, in time order)

The first thing to run when an alert fires. One readable line per event: process started, connection made, file written, registry changed, DNS lookup, download mark. Set the host and the time range, and add a ProcessGuid filter to focus on one process.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" host="YOUR-HOST"
      earliest="10/11/2026:09:00:00" latest="10/11/2026:11:00:00"
(EventCode=1 OR EventCode=3 OR EventCode=11 OR EventCode=13 OR EventCode=15 OR EventCode=22 OR EventCode=23)
| eval what=case(EventCode=1,"PROCESS ".CommandLine, EventCode=3,"CONNECT ".DestinationIp.":".DestinationPort,
      EventCode=11,"FILE ".TargetFilename, EventCode=13,"REGISTRY ".TargetObject, EventCode=15,"DOWNLOAD MARK
      ".TargetFilename, EventCode=22,"DNS ".QueryName, EventCode=23,"FILE DELETED ".TargetFilename)
| eval who=mvindex(split(Image,"\\"),-1)
| sort 0 _time
| table _time EventCode who what
```